ASSIGNMENT-9.5

ROLL NO: 2303A51360

BATCH NO: 29

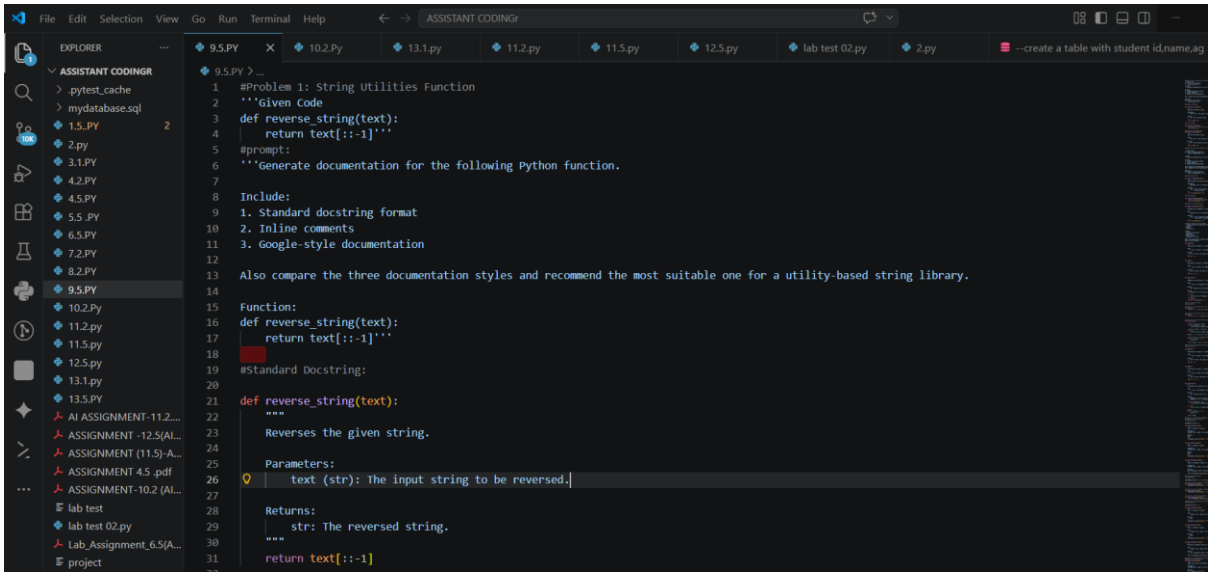
TASK-1:

Reverse a String

PROMPT:

Create a function to add two numbers and document it using different documentation styles.

CODE:



```
#Problem 1: String Utilities Function
'''Given Code
def reverse_string(text):
    return text[::-1]'''
#prompt:
'''Generate documentation for the following Python function.

Include:
1. Standard docstring format
2. Inline comments
3. Google-style documentation

Also compare the three documentation styles and recommend the most suitable one for a utility-based string library.

Function:
def reverse_string(text):
    return text[::-1]'''

#Standard Docstring:

def reverse_string(text):
    """
    Reverses the given string.

    Parameters:
        text (str): The input string to be reversed.

    Returns:
        str: The reversed string.
    """
    return text[::-1]
```

OBSERVATION:

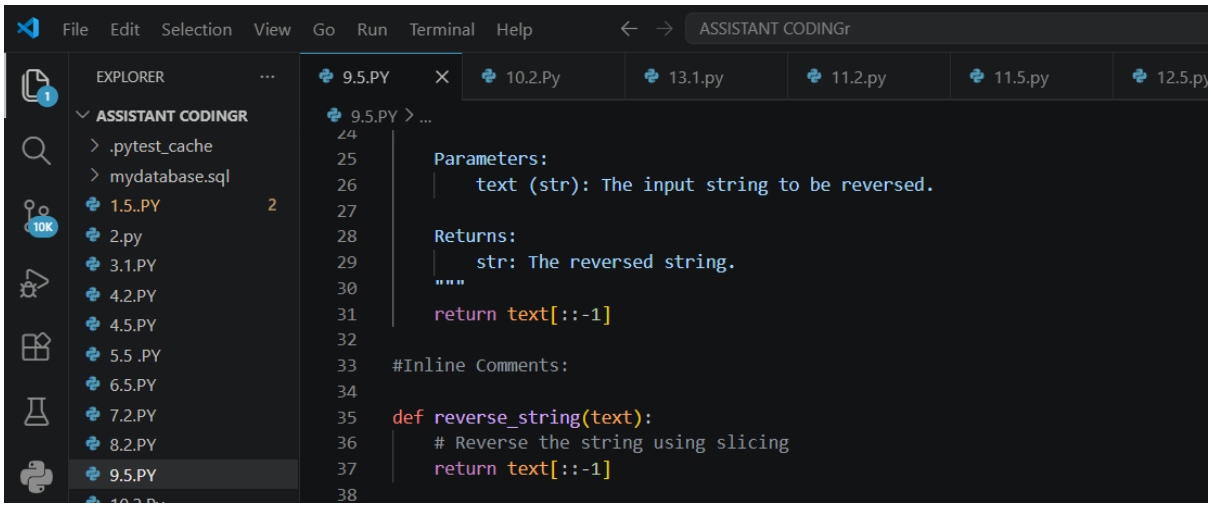
- The function reversed the string correctly.
- The docstring explained parameters and return value.
- It provided basic documentation.
- Structure is simple but less detailed.

Inline Comments

PROMPT:

Write a Python function to reverse a string using inline comments.

CODE:



OBSERVATION:

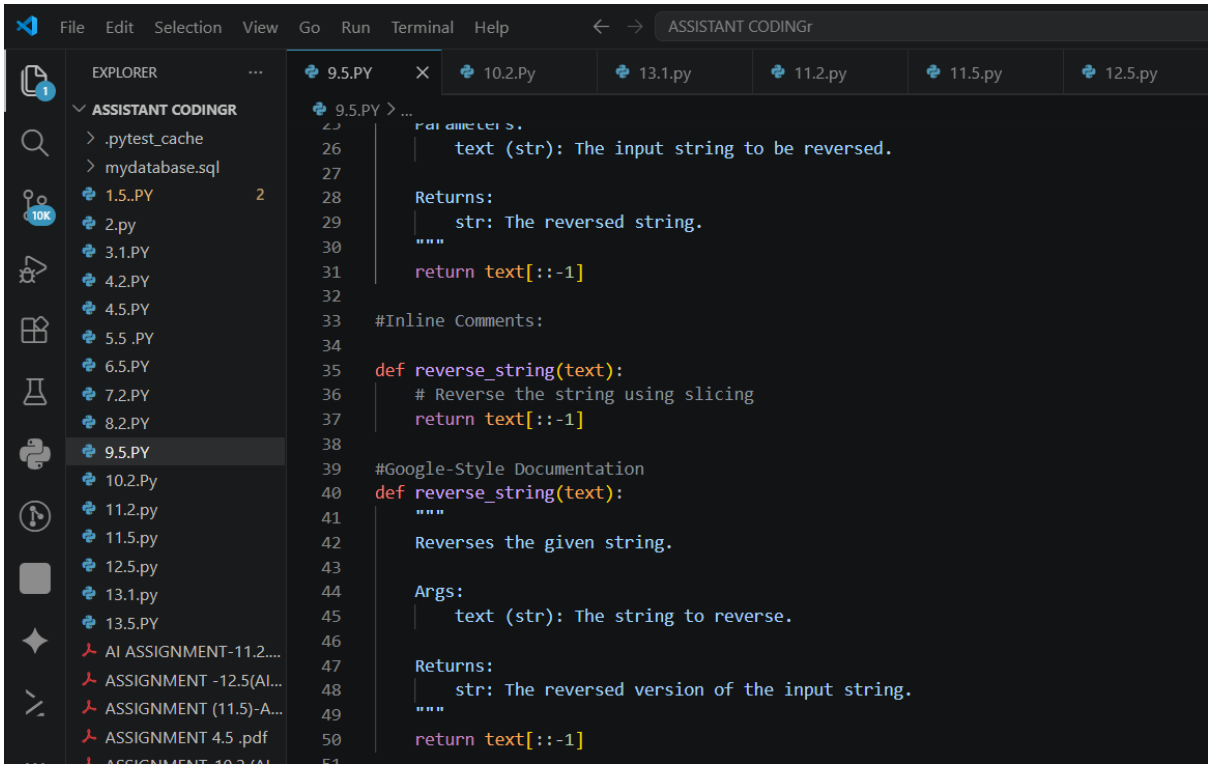
- The logic was easy to understand.
- Comments explained the slicing operation.
- Suitable for beginners.
- Not useful for automatic documentation tools.

Google Style Documentation

PROMPT:

Write a Python function to reverse a string using Google-style documentation.

CODE:



OBSERVATION:

- The documentation was clearly structured.
- Separate sections for Args and Returns.
- More professional format.
- Suitable for large projects.

TASK-2:

Factorial Program

#Standard Docstring

PROMPT:

Write a factorial function using Standard Docstring format.

CODE:

```
#Problem 2: Password Strength Checker

'''Given Code:
def check_strength(password):
    return len(password) >= 8'''

#prompt:
'''Generate documentation for the following Python function.

Include:
1. Standard docstring format
2. Inline comments
3. Google-style documentation

Also compare the documentation styles specifically for security-related code
and recommend the most appropriate style.

Function:
def check_strength(password):
    return len(password) >= 8'''

#Standard Docstring:

def check_strength(password):
    """
    Checks whether the given password meets minimum length requirements.

    Parameters:
        password (str): The password to validate.

    Returns:
        bool: True if the password length is at least 8 characters,
              otherwise False.
    """
    return len(password) >= 8
```

OBSERVATION:

- Parameters and return values were defined.
- Clear explanation of function purpose.
- Easy to understand but minimal structure.

#Inline Comments

PROMPT:

Write a factorial program using inline comments.

CODE:

```
#Inline Comments:
    (function) def check_strength(password: Any) -> bool
def check_strength(password):
    # Check if the password length is greater than or equal to 8
    return len(password) >= 8
```

OBSERVATION:

- Step-by-step multiplication was explained.
- Helps understand loop logic.

- Simple but not formally structured.

#Google Style

PROMPT:

Write a factorial function using Google-style documentation.

CODE:

```
#Google-Style Documentation:

def check_strength(password):
    """
    Validates the strength of a password based on minimum length.

    Args:
        password (str): The password string to validate.

    Returns:
        bool: True if password length is greater than or equal to
              8 characters, otherwise False.
    """
    return len(password) >= 8
```

OBSERVATION:

Clearly defined Args and Returns.

Looks professional and organized.

Better for collaborative projects.

TASK-3:

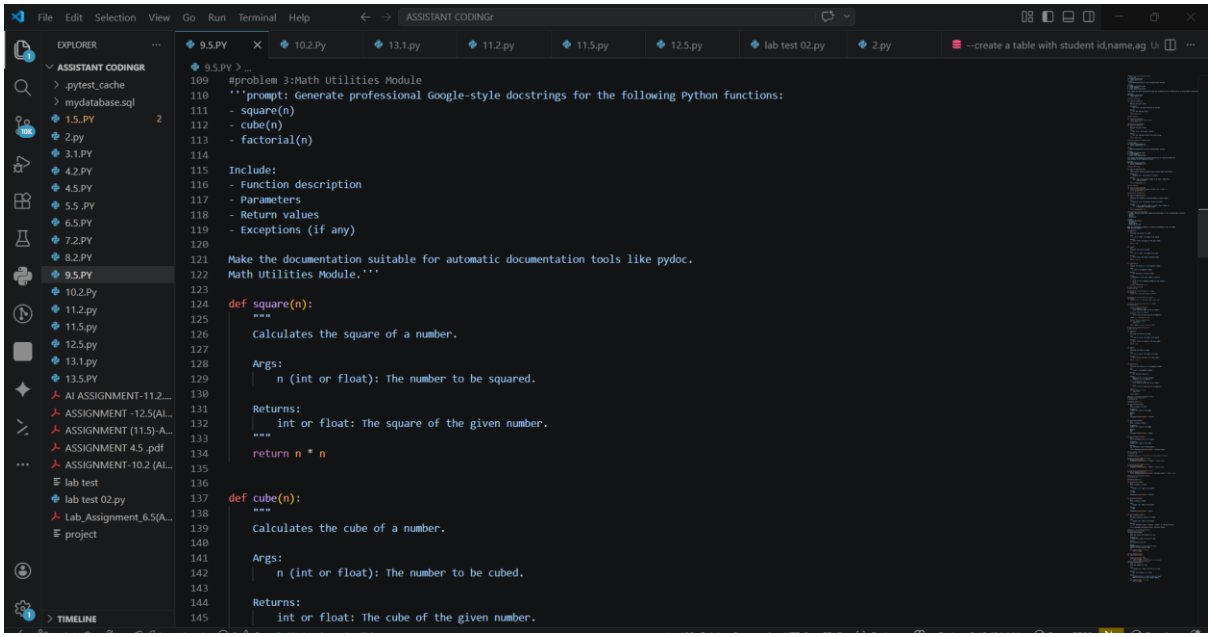
Calculator Function

Standard Docstring

PROMPT:

Write an addition function using Standard Docstring documentation.

CODE:



```python
#problem 3:Math Utilities Module
'''prompt: Generate professional Google-style docstrings for the following Python functions:
- square(n)
- cube(n)
- factorial(n)

Include:
- Function description
- Parameters
- Return values
- Exceptions (if any)

Make the documentation suitable for automatic documentation tools like pydoc.
Math Utilities Module.'''

def square(n):
    """
    Calculates the square of a number.

    Args:
        n (int or float): The number to be squared.

    Returns:
        int or float: The square of the given number.
    """
    return n * n


def cube(n):
    """
    Calculates the cube of a number.

    Args:
        n (int or float): The number to be cubed.

    Returns:
        int or float: The cube of the given number.
```

OBSERVATION:

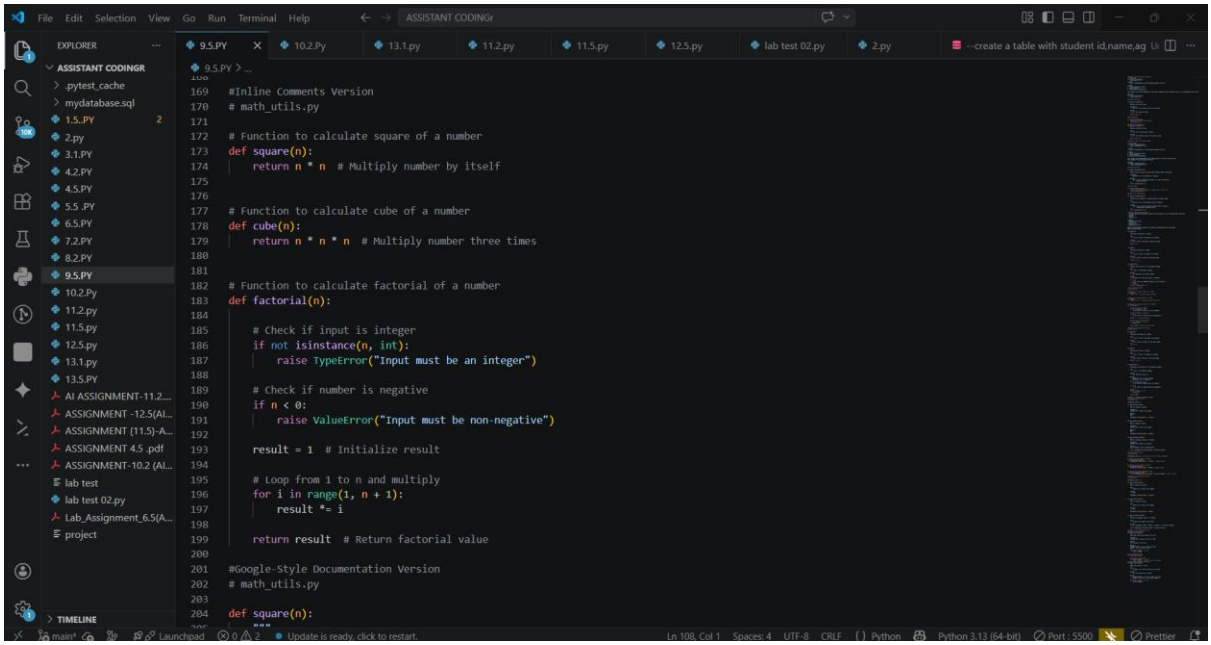
- Clearly mentioned parameters.
- Simple explanation of output.
- Basic documentation structure.

Inline Comments

PROMPT:

Write an addition function using inline comments.

CODE:



```python
#Inline Comments Version
# math_utils.py

# Function to calculate square of a number
def square(n):
    return n * n  # Multiply number by itself


# Function to calculate cube of a number
def cube(n):
    return n * n * n  # Multiply number three times


# Function to calculate factorial of a number
def factorial(n):

    # Check if input is integer
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")

    # Check if number is negative
    if n < 0:
        raise ValueError("Input must be non-negative")

    result = 1  # Initialize result

    # Loop from 1 to n and multiply
    for i in range(1, n + 1):
        result *= i

    return result  # Return factorial value

#Google-Style Documentation Version
# math_utils.py

def square(n):
    """
```

OBSERVATION:

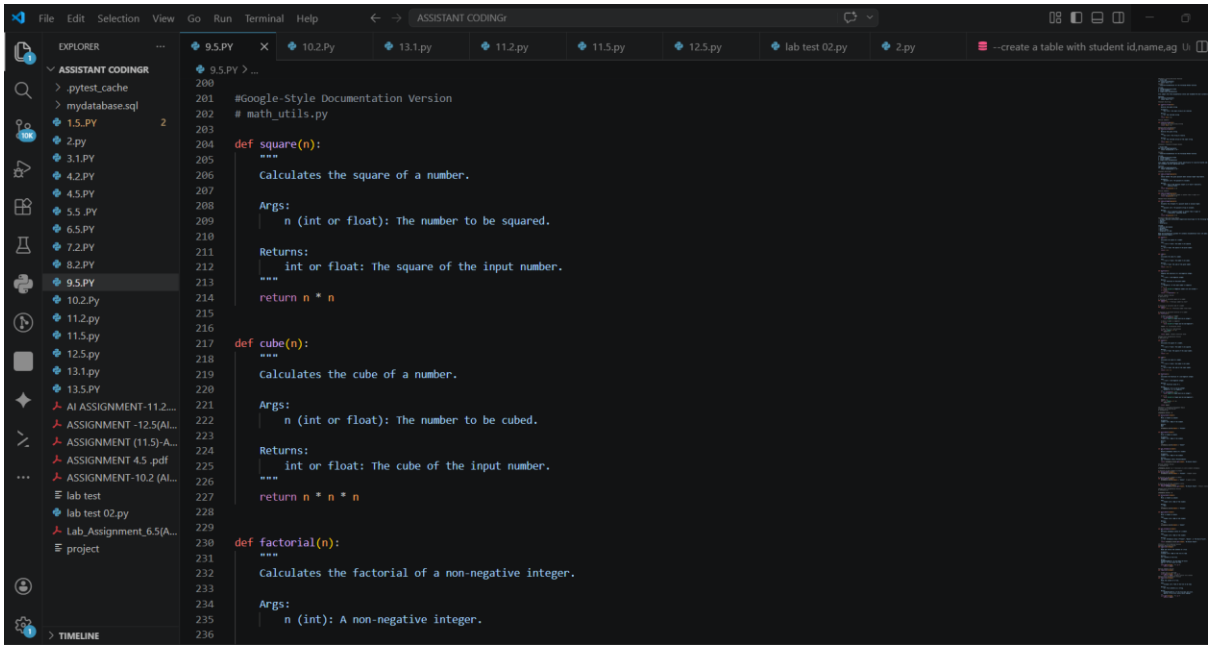
- Logic is very clear.
- Good for small programs.
- Does not describe function formally.

Google Style

PROMPT:

Write an addition function using Google-style documentation.

CODE:



OBSERVATION:

Structured format with Args and Returns.

Easy to maintain.

Professional documentation style.

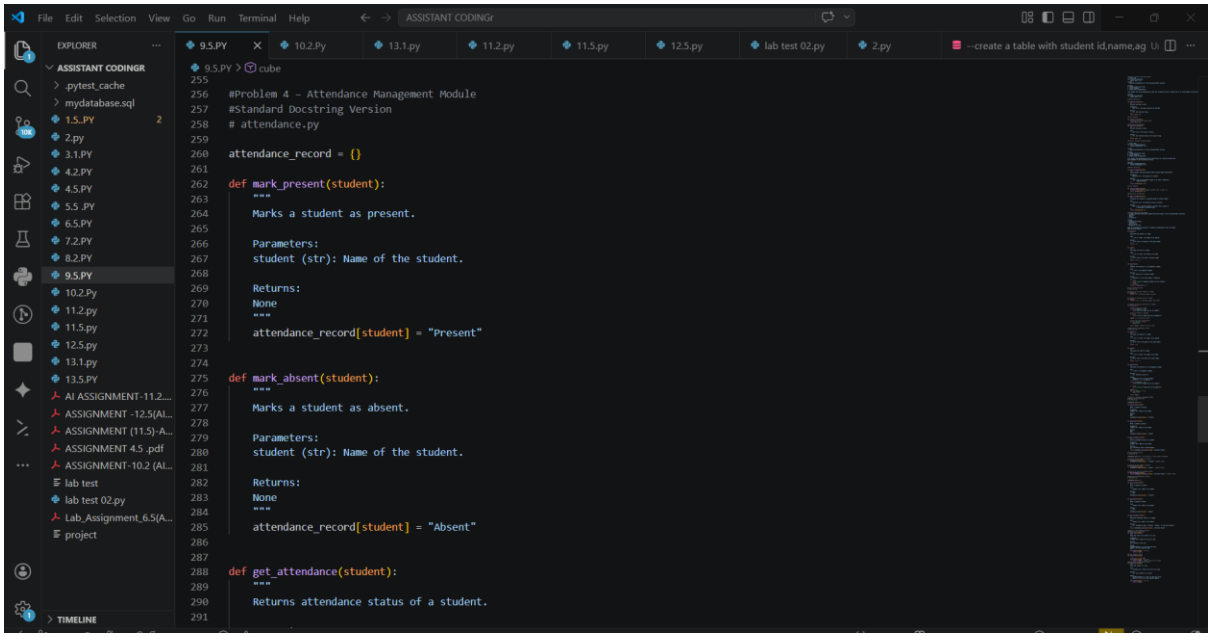
TASK-4:

#Standard Docstring

PROMPT:

Write an attendance management function using Standard Docstring format.

CODE:



```python
#Problem 4 – Attendance Management Module
#Standard Docstring Version
# attendance.py

attendance_record = {}

def mark_present(student):
    """
    Marks a student as present.

    Parameters:
    student (str): Name of the student.

    Returns:
    None
    """
    attendance_record[student] = "Present"


def mark_absent(student):
    """
    Marks a student as absent.

    Parameters:
    student (str): Name of the student.

    Returns:
    None
    """
    attendance_record[student] = "Absent"


def get_attendance(student):
    """
    Returns attendance status of a student.
```

OBSERVATION:

- Function purpose was clearly described.
- Parameters mentioned properly.
- Useful for small academic projects.

Inline Comments

PROMPT:

Write attendance management code using inline comments.

CODE:

```python
#Inline Comments Version
# attendance.py

attendance_record = {}  # Dictionary to store student attendance

# Function to mark student as present
def mark_present(student):
    attendance_record[student] = "Present"  # Update status


# Function to mark student as absent
def mark_absent(student):
    attendance_record[student] = "Absent"  # Update status


# Function to retrieve attendance status
def get_attendance(student):
    return attendance_record.get(student, "No Record Found")  # Return status
```

OBSERVATION:

Dictionary usage was explained.

Easy to understand program flow.

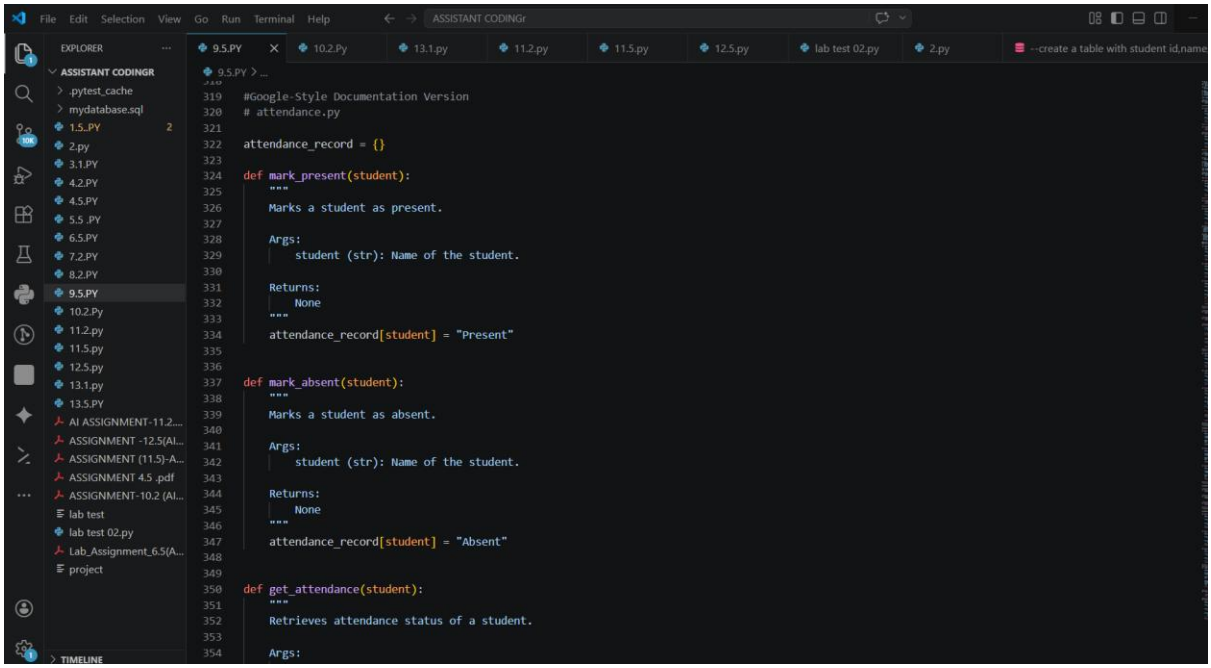
Limited documentation structure.

Google Style

PROMPT:

Write attendance management code using Google-style documentation.

CODE:



```python
#Google-Style Documentation Version
# attendance.py

attendance_record = {}

def mark_present(student):
    """
    Marks a student as present.

    Args:
        student (str): Name of the student.

    Returns:
        None
    """
    attendance_record[student] = "Present"


def mark_absent(student):
    """
    Marks a student as absent.

    Args:
        student (str): Name of the student.

    Returns:
        None
    """
    attendance_record[student] = "Absent"


def get_attendance(student):
    """
    Retrieves attendance status of a student.

    Args:
```

OBSERVATION:

- Clearly separated documentation sections.
- More readable and maintainable.
- Suitable for real-world applications.

TASK-5:

PROMPT:

Write a file reading function using Standard Docstring format.

CODE:

```python
#Problem 5 – File Handling Function
#Standard Docstring Version
def read_file(filename):
    """
    Reads and returns the contents of a file.

    Parameters:
    filename (str): Name of the file to read.

    Returns:
    str: Contents of the file.

    Raises:
    FileNotFoundError: If file does not exist.
    IOError: If file cannot be read.
    """
    with open(filename, 'r') as f:
        return f.read()
```

OBSERVATION:

Function purpose explained clearly.

Parameters and return values mentioned.

Exception handling not deeply structured

Inline Comments

PROMPT: Write a file reading function using inline comments.

CODE:

```python
#Inline Comments Version
def read_file(filename):

    # Open file in read mode
    with open(filename, 'r') as f:
        return f.read()  # Return entire file content
#Google Style Documentation Version
```

OBSERVATION:

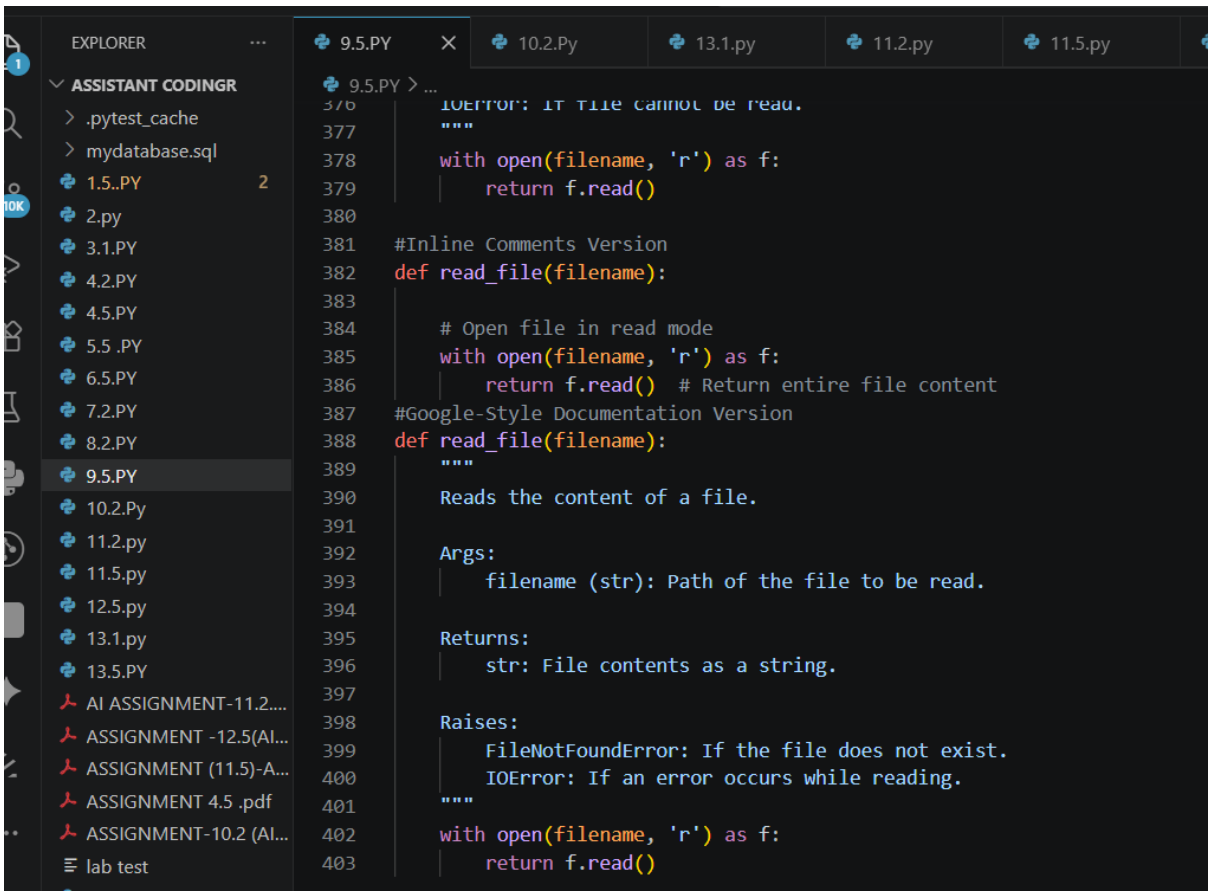
- File opening and reading steps explained.
- Simple and beginner-friendly.
- No formal documentation structure.

Google Style

PROMPT:

Write a file reading function using Google-style documentation including exception handling.

CODE:



OBSERVATION:

- Args, Returns, and Raises clearly defined.
- Exception handling documented properly.
- Best format for professional projects.
- Compatible with documentation tools.